



OptiCrop

Smart Agricultural Production Optimization Engine

AI-Powered Crop Recommendation Using Soil Nutrients, Weather Conditions & Environmental Analysis

Project Description:

OptiCrop is an advanced, data-driven software system designed to optimize agricultural production for a wide variety of crops. By integrating key environmental and soil factors **Nitrogen (N), Phosphorous (P), Potassium (K), temperature, humidity, pH, and rainfall**, OptiCrop analyzes crop-environment relationships to deliver intelligent, real-time crop recommendations that help farmers maximize yield and resource efficiency.

The platform includes a **Crop Recommendation Engine**, which predicts the most suitable crop for a given set of soil and climate conditions, a **Confidence Scoring module**, which quantifies how strongly the model supports its top prediction, and a **Top Crop Matches module**, which ranks and visualizes alternative crops the model considers well-suited to the same conditions.

Utilizing a **trained Machine Learning model (Random Forest Classifier)** built on historical agricultural data, OptiCrop processes the farmer's soil and weather inputs to deliver fast, reliable crop recommendations. Built with **Flask** and deployed publicly on **Render**, the platform ensures a seamless, responsive, and user-friendly experience. With clean input validation and a lightweight prediction pipeline, OptiCrop empowers farmers, agricultural researchers, and policymakers to make evidence-based farming decisions with confidence.

The knowledge and insights gained through OptiCrop can be used by agricultural researchers, policymakers, and stakeholders to advance the understanding of crop-environment interactions and inform the development of evidence-based agricultural strategies, ultimately contributing to more sustainable farming practices.

Scenarios

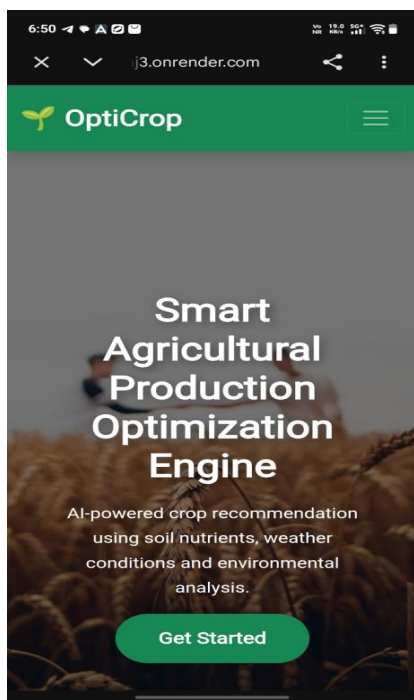
Scenario 1: A farmer enters soil and environmental details such as Nitrogen (N), Phosphorous (P), Potassium (K), temperature, humidity, pH level, and rainfall. The system analyzes the data and recommends the most suitable crop for maximum yield and better farming efficiency, along with a confidence score for the prediction.

Scenario 2: A user wants to understand whether the current soil and climate conditions are suitable for a particular crop. The application evaluates the input parameters and provides insights about crop compatibility, environmental suitability, and productivity potential, displaying the top matching crops ranked by confidence.

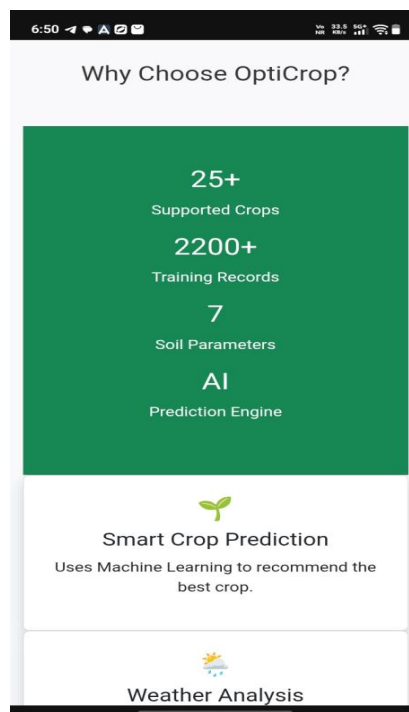
Scenario 3: An agricultural researcher or policymaker uses the system to analyze crop-environment relationships and identify patterns in agricultural production. The platform helps in making data-driven decisions for sustainable farming strategies and resource optimization.

Application Preview

The screenshots below show the deployed OptiCrop application: farmers enter soil and climate parameters, and the system returns a recommended crop with a confidence score and a chart of the top matching crops.



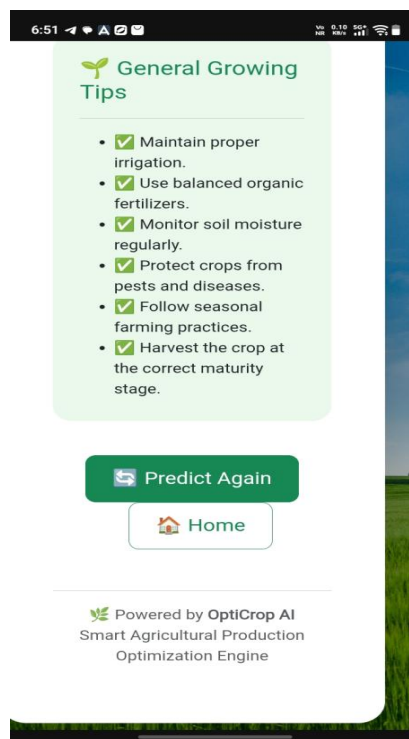
Input form — soil & climate parameters



Input form — additional fields & submit

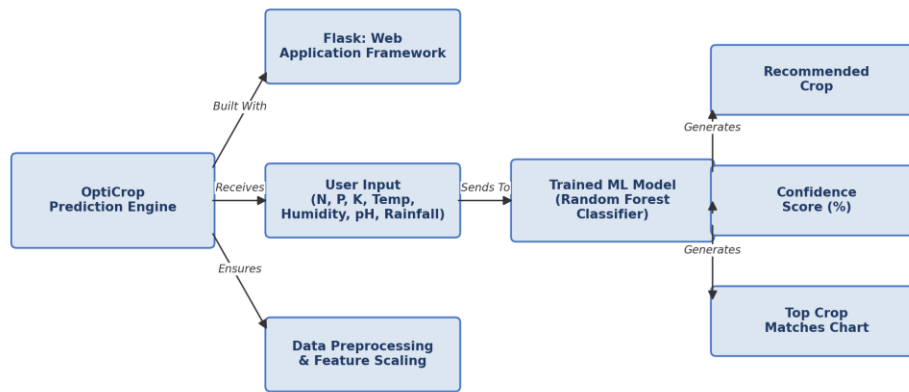
The screenshot shows the "Crop Recommendation" input form. It has a white background with a green header. The form contains several input fields for "Nitrogen (N)", "Phosphorous (P)", "Potassium (K)", "Temperature", "Humidity", "pH", and "Rainfall". A green "Predict Crop" button is at the bottom.

Recommended crop with confidence score



Top crop matches chart

Technical Architecture:

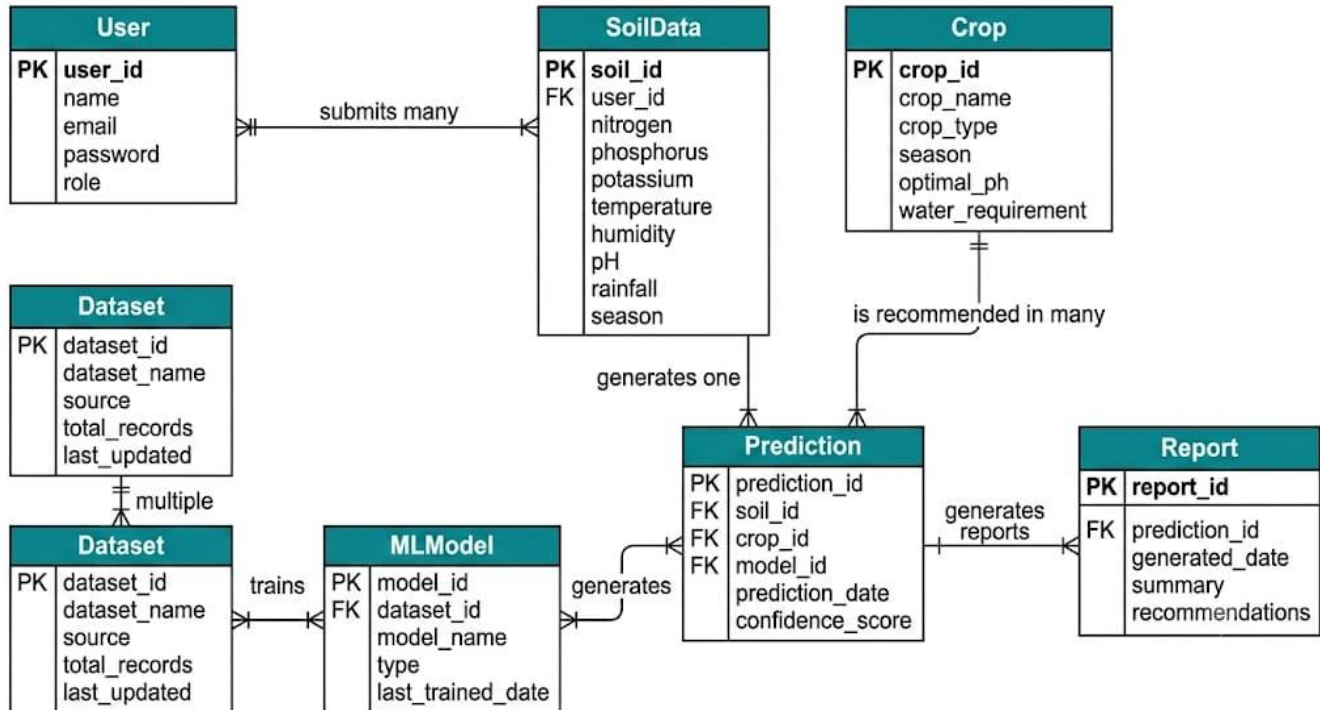


Description: OptiCrop utilizes a modular three-tier architecture to deliver fast, ML-driven crop recommendations. The frontend provides a responsive input form built with HTML, CSS, and JavaScript, while the Flask-based backend validates and preprocesses user input (soil nutrients, temperature, humidity, pH, and rainfall). It interfaces with a pre-trained Random Forest classification model to generate a predicted crop along with class probabilities, with deployment managed via Flask and exposed publicly through Render.

Pre-requisites:

- **Flask Framework Knowledge:** [Flask Documentation](#)
- **Machine Learning Fundamentals (scikit-learn):** [scikit-learn Documentation](#)
- **Pandas & NumPy for Data Handling:** [Pandas Documentation](#)
- **HTML, CSS, and JavaScript Skills:** [W3Schools HTML/CSS/JavaScript Tutorials](#)
- **Python Programming Proficiency:** [Python Documentation](#)
- **Version Control with Git:** [Git Documentation](#)
- **Development Environment Setup:** [Flask Installation Guide](#)
- **Render for Public Deployment:** [Render Documentation](#)

Entity Relationship Diagram:



Project Workflow:

Milestone 1: Dataset Preparation and Model Selection

- Activity 1.1:** Collect and explore the agricultural dataset containing N, P, K, temperature, humidity, pH, rainfall, and crop-label columns.
- Activity 1.2:** Clean the dataset, handle missing values, and perform exploratory data analysis (EDA) to understand feature-crop relationships.
- Activity 1.3:** Split the data into training and testing sets, and select an appropriate classification algorithm for crop prediction.
- Activity 1.4:** Set up the development environment, installing necessary libraries and dependencies for Flask, scikit-learn, and pandas.

Milestone 2: Model Training and Evaluation

- Activity 2.1:** Train a Random Forest Classifier on the preprocessed dataset and tune hyperparameters for optimal accuracy.
- Activity 2.2:** Evaluate model performance using accuracy, precision, recall, and a confusion matrix, then serialize the trained model using pickle/joblib.

Milestone 3: App.py Development

- Activity 3.1:** Write the main application logic in app.py, establishing routes for input handling, loading the trained model, and returning predictions.

Milestone 4: Frontend Development

- Activity 4.1:** Design and develop the input form using HTML, CSS, and JavaScript, ensuring a responsive and intuitive layout for entering soil and climate data.
- Activity 4.2:** Create dynamic templates with Flask's render_template to display the recommended crop, confidence score, and top crop matches chart.

Milestone 5: Deployment

- **Activity 5.1:** Prepare the application for deployment by configuring requirements.txt, the server environment, and ensuring all dependencies are correctly installed.
- **Activity 5.2:** Deploy the application publicly using Render to expose the Flask server over a secure public URL.

Milestone 6: Conclusion

Milestone 1: Dataset Preparation and Model Selection

In this milestone, we focus on preparing the agricultural dataset and selecting the appropriate machine learning model for OptiCrop's crop recommendation needs. This involves exploring the dataset's features, understanding crop-environment relationships, and choosing a model that balances prediction accuracy with speed.

Activity 1.1: Collect and Explore the Dataset

OptiCrop is trained on an agricultural dataset containing the following features: **Nitrogen (N), Phosphorous (P), Potassium (K), temperature, humidity, pH, and rainfall**, mapped to their corresponding recommended crop labels (e.g., rice, maize, coffee, jute, cotton).

- **Step 1 — Source the Dataset:** Obtain a crop recommendation dataset (e.g., from Kaggle) containing soil nutrient and climate readings alongside the ideal crop label.
- **Step 2 — Load and Inspect:** Load the dataset with pandas and inspect data types, ranges, and class distribution across crops.

Activity 1.2: Data Cleaning and Preprocessing

- **Step 1 — Handle Missing Values:** Check for and address any missing or inconsistent entries in the dataset.
- **Step 2 — Feature Scaling:** Apply scaling/normalization to numerical features so the model treats all inputs proportionally.
- **Step 3 — Encode Labels:** Encode the crop labels for use in classification, keeping a mapping back to crop names for display.

Activity 1.3: Select the Optimal Model

- **Evaluate Candidate Models:** Compare algorithms such as Decision Tree, Random Forest, and SVM based on accuracy and inference speed.
- **Select the Optimal Model:** Choose a Random Forest Classifier for its strong accuracy on tabular agricultural data and its ability to output class probabilities for the confidence score and top-matches chart.

Activity 1.4: Set Up the Development Environment

- **Install Python and Pip:** Ensure Python 3.8+ is installed on your machine along with pip for dependency management.
- **Create a Virtual Environment:** Set up a virtual environment using venv to isolate project dependencies.
- **Install Flask:** Use pip to install Flask.
- **Install ML Libraries:** Install scikit-learn, pandas, and numpy for data handling and model training.
- **Set Up Application Structure:** Create the project directory structure including folders for templates, static files (CSS, JS), the trained model file (model.pkl), and main application files (app.py, requirements.txt).

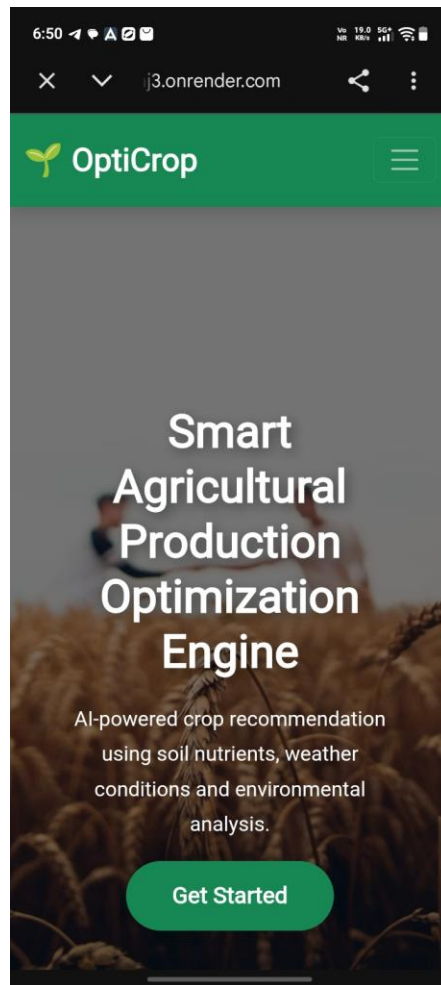


Fig. 1 — OptiCrop landing page

Project Overview

OptiCrop — the Smart Agricultural Production Optimization Engine — is an AI-powered web application that helps farmers, researchers, and policymakers make data-driven crop decisions. By analyzing seven key soil and environmental parameters — Nitrogen (N), Phosphorous (P), Potassium (K), Temperature, Humidity, pH, and Rainfall — OptiCrop's machine learning engine recommends the most suitable crop from a catalogue of 25+ supported crops, trained on a dataset of 2200+ agricultural records.

The platform is designed to improve yield, optimize resource use, and support sustainable farming practices, while also serving as a research tool for understanding crop-environment interactions.

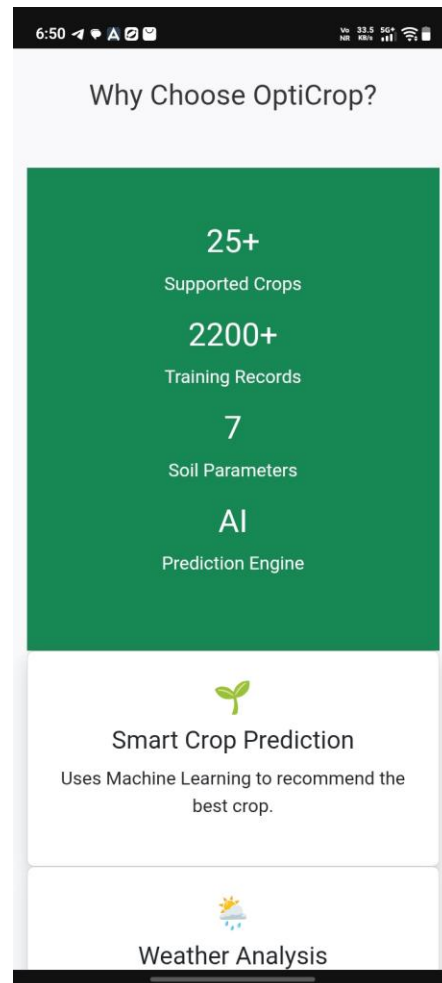


Fig. 2 — Why Choose OptiCrop: model stats and feature highlights

Key Use-Case Scenarios

- **Scenario 1 — Smart Crop Recommendation for Farmers:** A farmer enters soil and environmental details (N, P, K, temperature, humidity, pH, rainfall). The system analyzes the data and recommends the most suitable crop for maximum yield and better farming efficiency.
- **Scenario 2 — Crop Suitability & Environmental Assessment:** A user checks whether current soil and climate conditions are suitable for a particular crop, receiving insights on compatibility, environmental suitability, and productivity potential.
- **Scenario 3 — Agricultural Research & Policy Planning:** Researchers and policymakers use the platform to analyze crop-environment relationships, identify patterns, and inform evidence-based, sustainable agricultural strategies.

Milestone 2: Core Functionalities Development

Activity 2.1: Develop the Core Machine Learning Model

Implement the crop recommendation engine as a trained classification model driven by the seven soil and environmental input features.

- **Data Preparation:** Clean and preprocess the agricultural dataset (2200+ records covering 25+ crop labels), handling missing values and scaling numeric features (N, P, K, temperature, humidity, pH, rainfall).
- **Model Training:** Train a classification model (e.g. Random Forest / Decision Tree) on the seven input parameters to predict the optimal crop label, and evaluate accuracy using a train/test split.
- **Model Serialization:** Export the trained model (and any scaler/encoder) using pickle/joblib as model.pkl so it can be loaded at runtime by the Flask application.
- **Growing Tips Engine:** Attach a general growing-tips module that returns best-practice guidance (irrigation, fertilization, pest protection, harvesting) alongside every prediction.

Activity 2.2: Implement the Flask Backend

Define Routes in Flask

- Set up routes in app.py for the home page and the crop-prediction endpoint.

Process User Input

- Create a form in index.html for users to submit Nitrogen, Phosphorous, Potassium, Temperature, Humidity, pH, and Rainfall values.
- Use Flask's request.form / request.get_json() to retrieve the submitted data from the frontend.
- Validate that all seven fields are present and numeric before processing; return a 400 error with a clear message otherwise.

Integrate Model Prediction

- Within the /predict route handler, assemble the seven inputs into a feature array in the order expected by the trained model.
- Load the serialized model once at startup and call model.predict() on the incoming feature array.
- Map the predicted numeric/label output back to a human-readable crop name, then render it on the results page along with matching growing tips.

Milestone 3: App.py Development

Milestone 3 focuses on creating and refining the core logic of `app.py`, which serves as the backbone of the OptiCrop application. In this milestone, each route's functionality is established, the ML model is loaded and queried reliably, and the prediction pipeline is validated end-to-end.

Activity 3.1: Writing the Main Application Logic in `app.py`

Define the Core Routes in `app.py`

- Establish routes for OptiCrop's two endpoints, each serving a distinct purpose:
 - `/` — Home page route that renders the main `index.html` interface



- `/predict` — POST endpoint that accepts form data, runs the ML model, and returns the recommended crop



Set Up Route Handlers for Each Feature

- For the `/predict` route, implement logic that reads nitrogen, phosphorous, potassium, temperature, humidity, ph, and rainfall from the incoming form data.
- Apply input validation: re-render the form with an error message if any field is missing, empty, or non-numeric.
- Route form submissions from the frontend to the `/predict` endpoint, which processes the data and returns the results page.

Define the Prediction Helpers

- **FEATURE_ORDER list:** defines the exact order (N, P, K, temperature, humidity, ph, rainfall) the model expects, keeping the frontend form and backend inference in sync.



- **GENERAL_GROWING_TIPS list:** a curated set of best-practice tips (irrigation, organic fertilizers, soil-moisture monitoring, pest protection, seasonal practices, correct harvest timing) shown with every prediction.
- **load_model():** loads model.pkl, scaler.pkl, and the label encoder once at application startup, so /predict only performs fast in-memory inference on each request.

A screenshot of a mobile application interface for crop recommendation. The screen shows a form with seven input fields: Nitrogen (N), Phosphorous (P), Potassium (K), Temperature, Humidity, pH, and Rainfall. Each field is a simple text box. Below the fields is a green button labeled "Predict Crop". The app is running on a device with a status bar at the top showing the time 6:51 and various icons. The browser address bar shows "j3.onrender.com". The app title "Crop Recommendation" is displayed at the top of the form area, accompanied by a small green leaf icon.

Fig. 3 — Crop Recommendation input form (N, P, K, Temperature, Humidity, pH, Rainfall)

Milestone 4: Frontend Development

Milestone 4 focuses on developing a user-friendly, nature-themed interface for OptiCrop. This involves designing a clean, responsive HTML/CSS layout in a green agricultural palette, and creating dynamic content rendering so users can move smoothly from entering soil data to viewing their crop recommendation and growing tips.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure

- Develop the main HTML files (index.html, result.html) including a header with the OptiCrop logo and tagline, a hero section, an input form section, and a results section.
- Use semantic HTML elements to keep the structure organized and ensure the interface is accessible for all users.
- Integrate emoji/icon accents (🌱, 🌞, 🌧️) for section headings and Google Fonts for clean typography.

Design a Responsive Layout Using CSS

- Create a CSS stylesheet (static/css/style.css) implementing a green, nature-inspired theme with a hero background image and rounded card components.
- Use flexbox layouts to stack the stats banner (25+ crops, 2200+ records, 7 parameters, AI engine) and feature cards on the 'Why Choose OptiCrop' section.
- Add media queries to ensure the layout adapts across desktop, tablet, and mobile screen sizes.

Create Separate UI Components for Each Section

- **Input Form Card:** a single glass-style card holding labeled numeric fields for Nitrogen, Phosphorous, Potassium, Temperature, Humidity, pH, and Rainfall, with a prominent 'Predict Crop' button.
- **Result Card:** displays the recommended crop name prominently, followed by a General Growing Tips checklist card.
- **Action Buttons:** 'Predict Again' and 'Home' buttons let the user restart the flow or return to the landing page.

Activity 4.2: Creating Dynamic Content Rendering

Handle Form Submission

- Attach a submit handler to the crop-recommendation form that validates all seven fields are filled before submitting to /predict.
- During submission, disable the 'Predict Crop' button and show a loading state to prevent duplicate submissions.

Render Results Dynamically

- On the results page, render the predicted crop name and the General Growing Tips checklist returned by the backend.
- Provide 'Predict Again' and 'Home' navigation so users can quickly run another prediction or return to the landing page.

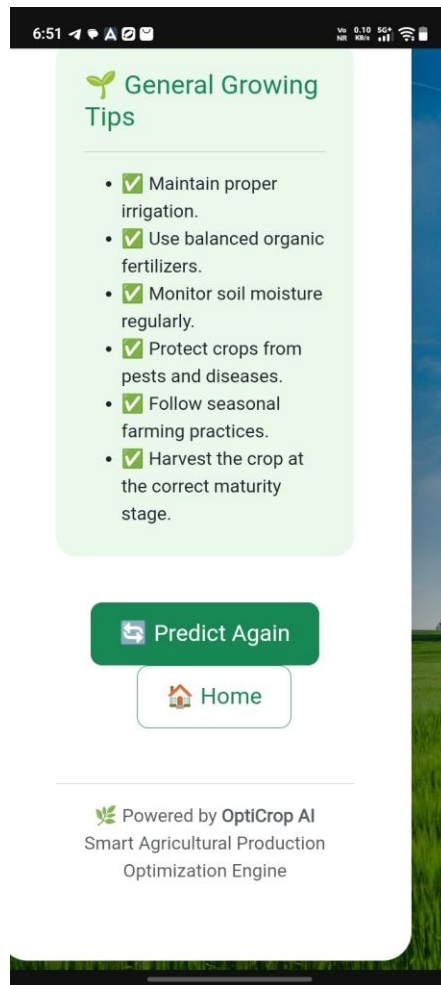


Fig. 4 — Prediction result with General Growing Tips, Predict Again & Home actions

Restore UI State After Prediction

- Re-enable the submit button and restore its original label and icon regardless of prediction success or failure. On validation errors, redisplay the form with the entered values preserved and a clear error message.

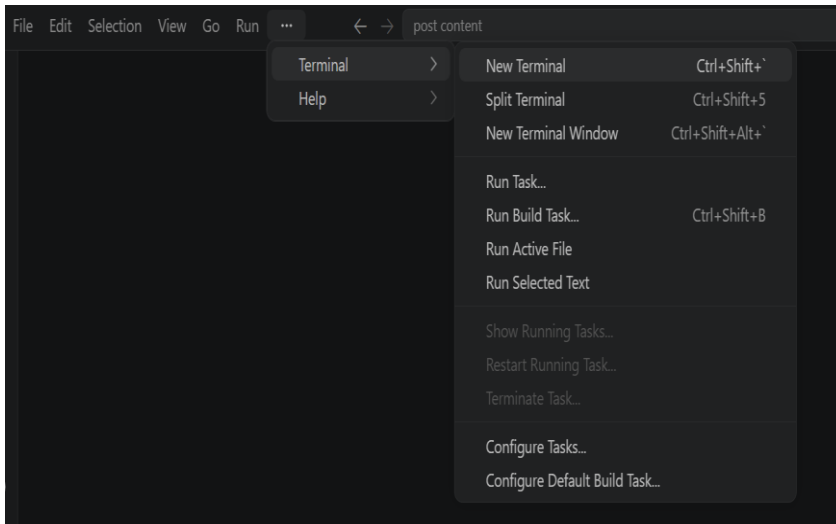
Milestone 5: Deployment

In Milestone 5, the focus is on deploying the OptiCrop application first locally to verify correct operation, and then publicly via Ngrok to share the app over a secure, accessible URL. This involves configuring the server environment, managing dependencies, and launching the crop recommendation engine for real-world use.

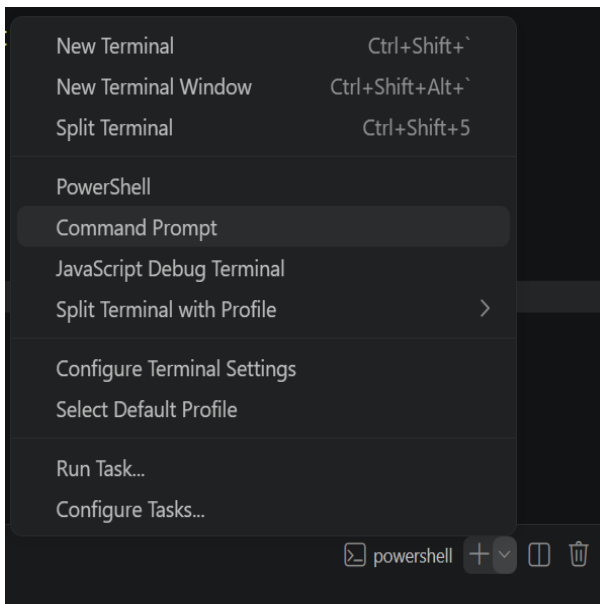
Activity 5.1: Preparing the Application for Local Deployment

Set Up a Virtual Environment:

- Open a New Terminal



- Then open "Command Prompt"



- Create and activate a virtual environment, then install all required dependencies (pandas, scikit-learn, Flask, numpy, joblib) from requirements.

```
D:\projects\OptiCrop>python -m venv myenv
D:\projects\OptiCrop>"d:\projects\OptiCrop\myenv\Scripts\activate"
(myenv) D:\projects\OptiCrop>pip install -r requirements.txt
```

Configure Environment Variables:

- Create a .env file in the project root and configure the trained model path and Flask settings. The app loads this automatically using python-dotenv at startup:

```
MODEL_PATH=model/crop_recommendation_model.pkl
SCALER_PATH=model/scaler.pkl
FLASK_ENV=development
```

Activity 5.2: Local Testing and Verification

Start the Flask Development Server:

- Run the application locally using:

```
(myenv) D:\projects\OptiCrop>python app.py
```

- Once running, open a browser and navigate to "http://127.0.0.1:5000" to access the app.

```
(myenv) D:\projects\OptiCrop>python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 118-427-963
```

- Interact with the crop recommendation engine: test with varied input combinations of Nitrogen (N), Phosphorous (P), Potassium (K), temperature, humidity, pH, and rainfall to verify the ML model's predictions are correctly generated, parsed, and rendered for different crop types.

Activity 5.3: Public Deployment via Ngrok

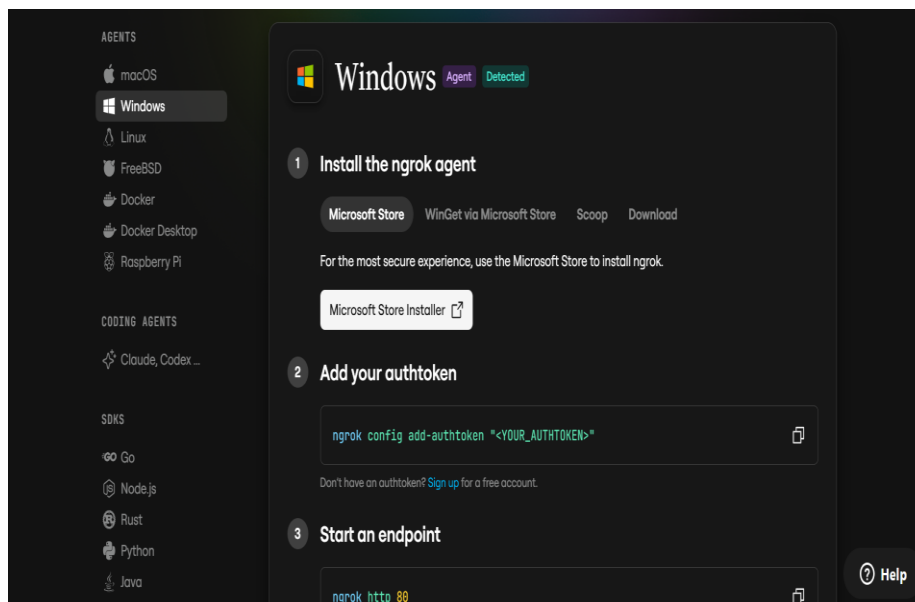
Ngrok creates a secure tunnel from a public URL to your local Flask server, making the OptiCrop app accessible from anywhere without a cloud hosting provider.

Install Ngrok and pyngrok:

- Install the pyngrok Python wrapper, which manages Ngrok programmatically within your Python project:

```
D:\projects\OptiCrop>pip install pyngrok
```

- Download and install the Ngrok client from <https://ngrok.com/download>.
- Click on "Microsoft Store Installer".



Configure Your Ngrok Authtoken:

- Sign up at ngrok.com and copy your authtoken from the Ngrok dashboard.

- The run_public.py script sets the auth token automatically using:

```
D:\projects\OptiCrop>ngrok.set_auth_token("YOUR_NGROK_AUTH_TOKEN")
```

- Replace the TOKEN value in run_public.py with your own Ngrok auth token before running.

Run the Public Deployment Script:

- Instead of running app.py directly, launch run_public.py to start both the Ngrok tunnel and the Flask server:

```
D:\projects\OptiCrop>python run_public.py
```

- The script opens an HTTP tunnel on port 5000 and prints the live public URL in the terminal:

```
=====
YOUR PUBLIC WEBSITE URL IS LIVE!
URL: https://xxxx-xx-xx-xxx-xx.ngrok-free.app
=====
```

- Share this URL with anyone who can access your OptiCrop app directly from their browser without any installation.

Important Notes:

- **debug=False** is required in run_public.py to prevent Flask from spawning a reloader process, which would interfere with Ngrok's tunnel management.
- The public URL changes each time you restart run_public.py (on the free Ngrok plan). For a persistent URL, upgrade to a paid Ngrok plan and configure a reserved domain.
- Ngrok free tier sessions expire after a few hours of inactivity. Restart run_public.py to get a new URL when this happens.

Exploring the Website's Web Pages

Home / Prediction Page:

OptiCrop
Smart Agricultural Production Optimization Engine

Soil Nutrient & Environmental Parameters

Nitrogen (N)	90 kg/ha
Phosphorous (P)	42 kg/ha
Potassium (K)	43 kg/ha
pH Level	6.5
Temperature (°C)	25.6
Humidity (%)	80.3
Rainfall (mm)	202.9

Predict Best Crop →

Description: The single-page interface of OptiCrop serves as both the landing page and the prediction tool. It features a dark, nature-inspired glassmorphism design with animated green background blobs, a header displaying the OptiCrop brand (leaf icon + logo), and a tagline. The main section contains the soil and environmental parameter input form and the AI-generated recommendation panel all on one seamless page.

Input Section:

Description: A glass-styled card containing the main prediction form. Users enter the seven core parameters — Nitrogen (N), Phosphorous (P), Potassium (K), Temperature, Humidity, pH level, and Rainfall — either manually or via slider/numeric fields, and click the Predict Best Crop button. While the ML model processes the request, the button transitions to an animated loading spinner.

Results Section:



Description: Revealed dynamically after prediction, this section displays the model output in a two-column grid. The left card shows the top recommended crop with a confidence score and a ranked list of alternative suitable crops. The right column contains two glass cards: one with resource-efficiency and cultivation tips, and one summarizing the submitted input parameters as tags. The viewport automatically smooth-scrolls to this section upon generating a prediction.

Conclusion

OptiCrop is a data-driven agricultural decision-support platform built with Flask and a machine-learning crop recommendation model, styled with a nature-inspired glassmorphism UI. By analyzing Nitrogen, Phosphorous, Potassium, soil temperature, humidity, pH, and rainfall, it instantly recommends the most suitable crop and assesses environmental suitability, helping farmers, researchers, and policymakers make evidence-based decisions. The project, which included data preprocessing, model training, and deployment via Ngrok, highlights how targeted machine learning and a structured framework can boost yields, resource efficiency, and sustainable farming practices, with strong potential for future scalability. Looking ahead, OptiCrop offers significant opportunities for expansion, such as integrating real-time weather API data, adding satellite/soil-sensor image analysis for field-level precision, and implementing user authentication to save farm profiles and prediction history. By continuously refining the underlying model and enhancing frontend accessibility, the platform is well-positioned to evolve from a specialized tool into a comprehensive smart-agriculture assistant for farmers, researchers, and policymakers of all scales.